

Taller 1 – Programación funcional en Elixir

En grupos de máximo tres personas, resuelva cada punto en Elixir aplicando buenas prácticas de programación, documentación (@moduledoc, @doc) y el módulo Util para leer e imprimir.

Reglas sobre el uso de herramientas de IA

Puede usar internet y asistentes de IA, con estas condiciones:

1. **Solo se aceptan los conceptos vistos en las guías 4, 5 y 6.** Si su solución usa algo no visto, debe reescribirla o justificar por escrito por qué es necesario. Esto incluye Enum, Regex, String.to_atom/1, try/rescue para validar entradas, recursión o mapas.
2. **Bitácora.** Por cada punto, escriba un prompt que haya usado y **al menos una cosa que la IA hizo mal, de forma innecesariamente compleja o por fuera de lo visto**, y cómo la corrigió. Si no usó IA, indíquelo.
3. Las preguntas de **Análisis** deben responderse con la salida real de **su** programa, copiada de la terminal.

Punto 1: Paquete turístico

Una agencia vende paquetes que combinan vuelo y alojamiento. Debe calcular el valor total de un paquete.

Vuelo:

Destino	Código	Tarifa
Bogotá	1	\$180.000
Medellín	2	\$150.000
Cartagena	3	\$220.000
San Andrés	4	\$280.000

La maleta de bodega cuesta \$45.000 y es opcional. Si el destino es San Andrés la maleta es obligatoria: se agrega automáticamente cuando el usuario no la pidió y no se cobra dos veces cuando sí la pidió.

Alojamiento:

Tarifa base por noche:

Noches	Tarifa por noche
1 a 2	\$120.000
3 a 5	\$100.000
6 a 30	\$85.000

Descuento por tipo de cliente: frecuente 20%, corporativo 15%, ocasional 0%. **Este descuento aplica únicamente sobre el alojamiento**, nunca sobre el vuelo.

Recargo por temporada, según el mes de la reserva: diciembre y enero 25%, junio y julio 10%, resto del año 0%. El recargo se calcula sobre el valor del alojamiento que queda después de aplicar el descuento.

Paquete:

- El seguro de viaje es opcional y cuesta \$12.000 **por noche**.
- Si el paquete tiene 3 noches o más, se aplica un descuento de paquete del 5% **sobre la suma del alojamiento y el seguro**. El vuelo y la maleta no reciben este descuento.

El programa debe:

1. Solicitar: destino (1 a 4), número de noches, tipo de cliente (texto: frecuente, corporativo u ocasional), mes (1 a 12), si desea maleta y si desea seguro. Las respuestas válidas son si y no, sin importar mayúsculas.
2. Validar que el destino esté entre 1 y 4, las noches entre 1 y 30, el mes entre 1 y 12 y que el tipo de cliente exista. Si algo falla, retorne `{:error, motivo}`.
3. Retornar, en caso de éxito, una tupla anidada con tres bloques:

```
{:ok, {  
  {tarifa_vuelo, valor_maleta, origen_maleta},  
  {tarifa_noche, subtotal_hotel, descuento, recargo},  
  {seguro, descuento_paquete, total}  
}}
```

donde `origen_maleta` es `:automatico`, `:solicitado` o `:no_aplica`.

4. Mostrar el detalle completo, indicando si algún cargo fue agregado automáticamente, o el mensaje de error.

Requisitos técnicos:

- Use **guards** en dos lugares: para determinar la tarifa por noche según el número de noches y para determinar si el destino obliga la maleta.
- Use **cond** para el porcentaje de recargo según el mes.
- Use **pattern matching sobre átomos** (:frecuente, :corporativo, :ocasional) para el descuento de cliente, y pattern matching para las tarifas base de cada destino.
- La función de la maleta debe retornar la tupla {valor, origen}.
- Encadene las validaciones con **with (opcional)**.
- Defina tarifas, cargos, porcentajes y códigos de destino como **atributos de módulo** (por ejemplo @san_andres 4). No use números sueltos en el código.
- La función que muestra el resultado debe recibir la tupla anidada completa y desarmarla con pattern matching en la cabecera.
- Maneje el dinero como float y formatee con dos decimales usando :erlang.float_to_binary/2, como se vio en la guía 5. Recuerde que Elixir muestra los valores grandes en notación científica (por ejemplo 4.2e5).
- Las funciones de cálculo deben ser puras; la lectura y la impresión van aparte.

Análisis:

1. **Casos de prueba.** Complete la tabla con la salida real de su programa:

Destino	Noches	Cliente	Mes	Maleta	Seguro	Total
4	2	ocasional	3	no	no	\$565000.0
4	5	frecuente	12	si	si	\$857000.0
3	3	corporativo	7	si	no	\$531475.0
2	1	ocasional	1	no	si	\$312000.0
1	4	Frecuente	12	no	no	\$560000.0
1	31	frecuente	5	no	no	# noches invalido
5	4	frecuente	12	si	si	# de destino invalido

```
PS C:\Users\manue\Desktop\elixir> elixir parque_turistico.exs
```

```
=== SISTEMA DE COTIZACIÓN DE PAQUETES TURÍSTICOS ===
```

```
Ingrese el código del destino (1: Bogotá, 2: Medellín, 3: Cartagena, 4: San Andrés): 4
```

```
Ingrese el número de noches (1 a 30): 2
```

```
Ingrese el tipo de cliente (frecuente, corporativo u ocasional): ocasional
```

```
Ingrese el mes del viaje (1 a 12): 3
```

```
¿Desea llevar maleta de bodega adicional? (si/no): no
```

```
¿Desea incluir seguro de viaje por noche? (si/no): no
```

```
vuelo y maleta:
```

- Tarifa Base Vuelo: \$280000.0
- Maleta de Bodega: \$45000.0 (Agregada automáticamente por política de destino)

```
alojamienot:
```

- Tarifa/Noche: \$120000.0
- Subtotal Hotel: \$240000.0
- descuento liente: -\$0.0
- Recargo Temporada: +\$0.0

```
Servicios adicionales:
```

- Seguro de Viaje: \$0.0
- descuento paquete (5%): -\$0.0

```
valor total: $565000.0
```

```
PS C:\Users\manue\Desktop\elixir> elixir parque_turistico.exs
```

```
=== SISTEMA DE COTIZACIÓN DE PAQUETES TURÍSTICOS ===
```

```
Ingrese el código del destino (1: Bogotá, 2: Medellín, 3: Cartagena, 4: San Andrés): 4
```

```
Ingrese el número de noches (1 a 30): 5
```

```
Ingrese el tipo de cliente (frecuente, corporativo u ocasional): frecuente
```

```
Ingrese el mes del viaje (1 a 12): 12
```

```
¿Desea llevar maleta de bodega adicional? (si/no): si
```

```
¿Desea incluir seguro de viaje por noche? (si/no): si
```

```
vuelo y maleta:
```

- Tarifa Base Vuelo: \$280000.0
- Maleta de Bodega: \$45000.0 (Solicitada por el usuario)

```
alojamienot:
```

- Tarifa/Noche: \$100000.0
- Subtotal Hotel: \$500000.0
- descuento liente: -\$100000.0
- Recargo Temporada: +\$100000.0

```
Servicios adicionales:
```

- Seguro de Viaje: \$60000.0
- descuento paquete (5%): -\$28000.0

```
valor total: $857000.0
```

```

PS C:\Users\manue\Desktop\elixir> elixir parque_turistico.exs

=== SISTEMA DE COTIZACIÓN DE PAQUETES TURÍSTICOS ===
Ingrese el código del destino (1: Bogotá, 2: Medellín, 3: Cartagena, 4: San Andrés): 3
Ingrese el número de noches (1 a 30): 3
Ingrese el tipo de cliente (frecuente, corporativo u ocasional): corporativo
Ingrese el mes del viaje (1 a 12): 7
¿Desea llevar maleta de bodega adicional? (si/no): si
¿Desea incluir seguro de viaje por noche? (si/no): no

vuelo y maleta:
  - Tarifa Base Vuelo: $220000.0
  - Maleta de Bodega: $45000.0 (Solicitada por el usuario)

alojamienot:
  - Tarifa/Noche: $100000.0
  - Subtotal Hotel: $300000.0
  - descuento liente: -$45000.0
  - Recargo Temporada: +$25500.0

Servicios adicionales:
  - Seguro de Viaje: $0.0
  - desciento paquete (5%):-$14025.0

valor total: $531475.0

```

```

PS C:\Users\manue\Desktop\elixir> elixir parque_turistico.exs

=== SISTEMA DE COTIZACIÓN DE PAQUETES TURÍSTICOS ===
Ingrese el código del destino (1: Bogotá, 2: Medellín, 3: Cartagena, 4: San Andrés): 2
Ingrese el número de noches (1 a 30): 1
Ingrese el tipo de cliente (frecuente, corporativo u ocasional): ocasional
Ingrese el mes del viaje (1 a 12): 1
¿Desea llevar maleta de bodega adicional? (si/no): no
¿Desea incluir seguro de viaje por noche? (si/no): si

vuelo y maleta:
  - Tarifa Base Vuelo: $150000.0
  - Maleta de Bodega: $0.0 (No incluida)

alojamienot:
  - Tarifa/Noche: $120000.0
  - Subtotal Hotel: $120000.0
  - descuento liente: -$0.0
  - Recargo Temporada: +$30000.0

Servicios adicionales:
  - Seguro de Viaje: $12000.0
  - desciento paquete (5%):-$0.0

valor total: $312000.0

```

```

PS C:\Users\manue\Desktop\elixir> elixir parque_turistico.exe

=== SISTEMA DE COTIZACIÓN DE PAQUETES TURÍSTICOS ===
Ingrese el código del destino (1: Bogotá, 2: Medellín, 3: Cartagena, 4: San Andrés): 1
Ingrese el número de noches (1 a 30): 4
Ingrese el tipo de cliente (frecuente, corporativo u ocasional): Frecuente
Ingrese el mes del viaje (1 a 12): 12
¿Desea llevar maleta de bodega adicional? (si/no): no
¿Desea incluir seguro de viaje por noche? (si/no): no

vuelo y maleta:
- Tarifa Base Vuelo: $180000.0
- Maleta de Bodega: $0.0 (No incluida)

alojamienot:
- Tarifa/Noche: $100000.0
- Subtotal Hotel: $400000.0
- descuento liente: -$80000.0
- Recargo Temporada: +$80000.0

Servicios adicionales:
- Seguro de Viaje: $0.0
- descuento paquete (5%): -$20000.0

valor total: $560000.0

```

```

PS C:\Users\manue\Desktop\elixir> elixir parque_turistico.exe

=== SISTEMA DE COTIZACIÓN DE PAQUETES TURÍSTICOS ===
Ingrese el código del destino (1: Bogotá, 2: Medellín, 3: Cartagena, 4: San Andrés): 1
Ingrese el número de noches (1 a 30): 31
Ingrese el tipo de cliente (frecuente, corporativo u ocasional): frecuente
Ingrese el mes del viaje (1 a 12): 5
¿Desea llevar maleta de bodega adicional? (si/no): no
¿Desea incluir seguro de viaje por noche? (si/no): no
Error al calcular el paquete: Número de noches inválido (debe ser de 1 a 30)
PS C:\Users\manue\Desktop\elixir> elixir parque_turistico.exe

=== SISTEMA DE COTIZACIÓN DE PAQUETES TURÍSTICOS ===
Ingrese el código del destino (1: Bogotá, 2: Medellín, 3: Cartagena, 4: San Andrés): 5
Ingrese el número de noches (1 a 30): 4
Ingrese el tipo de cliente (frecuente, corporativo u ocasional): frecuente
Ingrese el mes del viaje (1 a 12): 12
¿Desea llevar maleta de bodega adicional? (si/no): si
¿Desea incluir seguro de viaje por noche? (si/no): si
Error al calcular el paquete: Destino inválido (debe ingresar un número entre 1 y 4)
PS C:\Users\manue\Desktop\elixir> 

```

2. Orden de las operaciones. Tome el segundo caso de la tabla y calcule el total invirtiendo el orden: aplique primero el recargo de temporada y después el descuento de cliente. ¿Cambia el valor? Si cambia, indique en cuántos pesos y explique por qué. Si no cambia, demuestre por qué no puede cambiar. Luego responda: el descuento de paquete del 5% se calcula sobre el alojamiento y el

seguro, pero no sobre el vuelo. ¿En qué línea de su código queda expresado ese límite? ¿Qué tendría que cambiar si la agencia decidiera extenderlo al vuelo?

R= No cambia el valor final al invertir el orden de las operaciones, ya que al calcular el recargo de temporada (25%) y el descuento de cliente (20%) sobre el subtotal del alojamiento ($S = \$500.000$), se aplican los factores multiplicativos 1.25 y 0.80; por la **propiedad conmutativa y asociativa de la multiplicación** ($S \times 0.80 \times 1.25 = S \times 1.25 \times 0.80 = \500.000), la secuencia no afecta el resultado. El límite del descuento de paquete del 5% queda expresado en el código en la línea `base_paquete = alojamiento_final + monto_seguro`, y si la agencia decidiera extenderlo al vuelo, solo habría que modificar esa variable para incluir los costos de transporte: `base_paquete = tarifa_vuelo + valor_maleta + alojamiento_final + monto_seguro`.

3. Respuestas inesperadas. ¿Qué hace su programa si el usuario responde sí (con tilde), S o tal vez a la pregunta de la maleta? ¿Qué decisión de diseño tomó y por qué? Tenga en cuenta que para San Andrés la respuesta del usuario puede terminar siendo irrelevante.

R= Si el usuario responde `"sí"` (con tilde), `"S"` o `"tal vez"`, la función `validar_booleano/1` no reconoce el texto y el programa muestra un error diciendo `"Respuesta no válida (responda 'si' o 'no')"`, deteniendo todo el proceso sin hacer ningún cálculo. Tomé la decisión de hacer una validación sencilla pero estricta para evitar que entren respuestas extrañas que puedan confundir el programa o cobrar mal los precios. Aunque en el caso de San Andrés la respuesta del usuario al final no importe porque la maleta se cobra obligatoriamente por regla del viaje, pedir la palabra exacta `"si"` o `"no"` ayuda a que todas las preguntas del sistema se respondan de la misma forma, ordenada y sin enredos.

4. Cambio en la regla. La agencia anuncia que Cartagena también tendrá maleta obligatoria. ¿Cuántas líneas de su código debe cambiar? Si son más de una, reestructure su solución para que sea una sola y explique cómo lo logró.

R= Para hacer este cambio solo hay que modificar una sola línea de código. Esto se logró organizando la regla en una pequeña función auxiliar (`es_maleta_obligatoria?/1`) que utiliza *guards*; así, en lugar de repetir la lógica por todo el programa, únicamente se agrega la constante `@dest_cartagena` a la lista de destinos obligatorios en la cláusula de esa función (`when destino in [@dest_san_andres, @dest_cartagena]`), permitiendo que todo el cálculo y el flujo del sistema se adapten automáticamente sin tocar nada más.

5. Riesgo de String.to_atom/1. Investigue por qué convertir texto del usuario a átomo con esa función se considera un riesgo en Elixir. Explíquelo con sus palabras en máximo cinco líneas.

R= En Elixir, la función `String.to_atom/1` es peligrosa porque los átomos guardados en la memoria del sistema no se borran nunca y tienen un límite máximo. Ya que nunca se borran hasta el punto en el que el programa se llena hasta llegar al límite.

Punto 2: Validación de cupón promocional

Implemente un programa que valide un código de cupón con estas reglas:

- Debe tener al menos 10 caracteres.
- Debe contener al menos una letra mayúscula.
- Debe contener al menos un número.
- No debe contener espacios en blanco.

El programa debe retornar:

- `{:ok, "Cupón válido"}` si cumple todas las reglas.
- `{:error, mensaje}` con **todas** las reglas incumplidas, separadas por coma. Por ejemplo: `{:error, "debe tener al menos 10 caracteres, no debe contener espacios"}`.

Requisitos técnicos:

- Use `String.length/1`, `String.replace/3`, `String.downcase/1` y `String.contains?/2`.
- **Acumule los errores sin listas.** Cada validación recibe y retorna una tupla `{cupon, errores}`, donde errores es un texto que empieza vacío (`""`). Encadénalas con el pipe:

```
{cupon, ""}  
|> validar_longitud()  
|> validar_mayuscula()  
|> validar_numero()  
|> validar_espacios()  
|> construir_resultado()
```

- `construir_resultado/1` decide entre `{:ok, _}` y `{:error, _}` con pattern matching, no con if.

Análisis:

1. **Casos de prueba.** Pruebe estos cupones y copie la salida: `PROMO2026X`, `promo`, `Promo 2026`, `ABCDEFGHJIJ`, `abcdefghijkl1`, `DESCUENTO1` (con un espacio al final).

`PROMO2026X:`

Ingrese el codigo del cupon:`PROMO2026X`

Éxito: Cupón válido

`promo:`

Ingrese el codigo del cupon:promo

Error: debe tener al menos 10 caracteres, debe contener al menos una letra mayúscula, debe contener al menos un número

Promo 2026:

Ingrese el codigo del cupon:Promo 2026

Error: no debe contener espacios

ABCDEFGHJIJ:

Ingrese el codigo del cupon:ABCDEFGHJIJ

Error: debe contener al menos un número

abcdefghijkl1:

Ingrese el codigo del cupon:abcdefghijkl1

Error: debe contener al menos una letra mayúscula

DESCUENTO1 :

Ingrese el codigo del cupon:DESCUENTO1

Éxito: Cupón válido

2. **El espacio final.** El último cupón tiene un espacio al final. ¿Su programa lo detectó? Explique exactamente por qué, revisando el código de Util.leer/2. ¿Es un error del programa o un comportamiento deseable?

Mi programa no detectó el espacio al final del cupón DESCUENTO1 . Como se ve en la consola, lo marcó como cupón válido

Esto sucede por cómo funciona la función util.leer según vimos en la Guía 04, cuando usamos IO.gets para pedirle al usuario que escriba algo, elixir siempre captura un salto de línea como si fuera un espacio invisible al momento de presionar enter, para limpiar eso, la función util.leer usa por dentro otra herramienta llamada string.trim

El detalle está en que string.trim no solo borra el salto de línea, sino que también borra automáticamente cualquier espacio en blanco que el usuario haya dejado por accidente al principio o al final de lo que escribió por eso, cuando escribo DESCUENTO1 , la función le quita el espacio y mi programa solo revisa DESCUENTO1

Es un error o es deseable?

Es un comportamiento deseable. En la vida real, es muy común que las personas presionen la barra espaciadora por error al copiar y pegar un código promocional o antes de presionar enter. Si el

programa fuera tan estricto y fallara por un espacio accidental al final, los clientes se molestarían. Limpiar esos espacios basura antes de revisar el cupón hace que el programa sea más amigable de usar

3. **Orden de las validaciones.** Si cambia el orden de las validaciones en el pipe, ¿cambia el resultado? ¿Y el mensaje? ¿Qué le dice esto sobre la independencia de estas funciones?

Cambiar el orden de las validaciones no cambia el resultado, si un código es inválido lo seguirá siendo si falla al inicio o al final, es como una multiplicación, el orden de los factores no altera el resultado, y si el código es válido seguirá siendo válido así se evalúen otros factores al inicio o al final, el mensaje sí cambiaría ya que al evaluar otras cosas al inicio o al final se van acumulando según vaya fallando, pero el resultado al final será el mismo.

Esto también ayuda a entender que cada función es independiente, simplemente si falla, falla en esa parte y pasa su reporte en este caso el error, pero cada función es independiente y no afecta a las otras en el funcionamiento.

4. **Diferencia con with.** En algunos ejemplos de clase, vimos with. Explique por qué with no sirve para este punto y qué tiene su solución que lo hace diferente.

No se debe usar with en este ejemplo ya que con esa función el programa se detiene apenas encuentra el primer error, e informa solamente de ese error, y la idea de este programa es que nos informe de los errores que vaya teniendo y no solamente del primero.

Y mi solución lo que tiene de diferente al with es que sí o sí pasa por todas las validaciones hasta que da el resultado final y así nos da todos los errores como lo pide el ejemplo.

Para la función de validar_numero se debe de usar una lista al inicio que contenga los números desde el 0 hasta el 9 ya definida, ya que sino se usa, habría que preguntar con String.contains? y el código quedaría más largo y más propenso a errores y a poca modificación en dado caso y sino se usa quedaría así

```
defp contiene_numero?(cupon) do String.contains?(cupon, "0") or String.contains?(cupon, "1") or  
String.contains?(cupon, "2") or String.contains?(cupon, "3") or String.contains?(cupon, "4") or  
String.contains?(cupon, "5") or String.contains?(cupon, "6") or String.contains?(cupon, "7") or  
String.contains?(cupon, "8") or String.contains?(cupon, "9") end
```

Y usando la función que realice quedaría algo más sencillo como esto

```
@numeros "0" "1" "2" "3" "4" "5" "6" "7" "8" "9"
```

```
defp validar_numero({cupon, errores}) do  
  cupon_sin_numeros = String.replace(cupon, @numeros, "")
```

```

if String.length(cupon) == String.length(cupon_sin_numeros) do
  {cupon, agregar_errorerrores, "debe contener al menos un número"}}
else
  {cupon, errores}
end
end

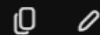
```

Y así quedaría con más recursividad y menos propenso a cometer errores en un futuro

Solo use la ia para la documentacion y para darme cuenta de que habria que usar una lista o sino el codigo se volvia muy largo y poco recursivo para saber si tenia numeros el cupon

Ayudame a hacer una documentacion muy sencilla para este codigo

defmodule Cupon do



Aquí tienes una documentación clara, corta y fácil de entender, aplicando el estándar de Elixir (usando `@moduledoc` y `@doc` como se muestra al principio de la Guía 04) para que la integres directamente en tu código.

Código con la documentación incluida

Punto 3: Cajero automático

Un banco requiere simular un retiro en cajero para una única cuenta con los siguientes datos, definidos como atributos de módulo:

```

@pin_correcto "0482"
@saldo_disponible 450_000
@limite_por_retiro 600_000

```

El programa solicita el PIN, el tipo de cuenta (ahorros o corriente) y el monto a retirar. Las validaciones deben hacerse **en este orden** y detenerse en la primera que falle:

1. El PIN debe tener exactamente 4 caracteres y coincidir con el PIN correcto.
2. El tipo de cuenta debe ser válido.
3. El monto debe ser mayor que cero.
4. El monto debe ser múltiplo de \$10.000.
5. El monto no puede superar el límite por retiro.
6. El monto **más la comisión** no puede superar el saldo disponible.

Comisión:

- Cuenta de ahorros: \$0 si el monto es menor o igual a \$200.000; \$2.500 si es mayor.
- Cuenta corriente: \$4.000 fijos.

Entrega de billetes. El cajero entrega el monto con la **menor cantidad posible** de billetes de \$50.000, \$20.000 y \$10.000.

Resultado. El programa debe retornar:

- `{:ok, {monto, comision, saldo_restante, {billetes_50, billetes_20, billetes_10}}}` si el retiro es posible.
- `{:error, motivo}` si alguna validación falla.

Finalmente, muestre un comprobante legible.

Requisitos técnicos:

- Use `with` con `bloque else` para encadenar las validaciones.
- Use *guards* para la comisión y *pattern matching* para el tipo de cuenta.
- El PIN se lee como `:string`, **no** como `:integer`.

Análisis:

1. **Casos de prueba.** Complete la tabla:

PIN	Cuenta	Monto	Resultado
0482	ahorros	150000	Comisión: \$0.0 Total debitado: \$150000.0 Saldo restante: \$300000.0 Billetes de 50.000: 3 Billetes de 20.000: 0 Billetes de 10.000: 0
0482	ahorros	80000	Comisión: \$0.0

PIN	Cuenta	Monto	Resultado
			Total debitado: \$80000.0 Saldo restante: \$370000.0 Billetes de 50.000: 1 Billetes de 20.000: 1 Billetes de 10.000: 1
0482	corriente	250000	Comisión: \$4000.0 Total debitado: \$254000.0 Saldo restante: \$196000.0 Billetes de 50.000: 5 Billetes de 20.000: 0 Billetes de 10.000: 0
0482	ahorros	450000	Monto a retirar: 450000 Error: Fondos insuficientes para cubrir el monto y la comisión
0482	ahorros	95000	Monto a retirar: 95000 Error: El monto debe ser múltiplo de \$10.000
1234	ahorros	700000	Ingrese su PIN: 1234 Tipo de cuenta (ahorros/corriente): ahorros Monto a retirar: 700000 Error: PIN incorrecto

PIN	Cuenta	Monto	Resultado
0482	nomina	50000	Tipo de cuenta (ahorros/corriente): nomina Monto a retirar: 50000 Error: Tipo de cuenta no válido

```
Ingrese su PIN: 0482
Tipo de cuenta (ahorros/corriente): ahorros
Monto a retirar: 150000
Monto retirado: $150000.0
Comisión: $0.0
Total debitado: $150000.0
Saldo restante: $300000.0
Billetes de 50.000: 3
Billetes de 20.000: 0
Billetes de 10.000: 0
```

```
PS C:\Users\manue\Desktop\elixir> elixir cajero.exe
Ingrese su PIN: 0482
Tipo de cuenta (ahorros/corriente): ahorros
Monto a retirar: 80000
Monto retirado: $80000.0
Comisión: $0.0
Total debitado: $80000.0
Saldo restante: $370000.0
Billetes de 50.000: 1
Billetes de 20.000: 1
Billetes de 10.000: 1
```

```
PS C:\Users\manue\Desktop\elixir> elixir cajero.exe
Ingrese su PIN: 0482
Tipo de cuenta (ahorros/corriente): corriente
Monto a retirar: 250000
Monto retirado: $250000.0
Comisión: $4000.0
Total debitado: $254000.0
Saldo restante: $196000.0
Billetes de 50.000: 5
Billetes de 20.000: 0
Billetes de 10.000: 0
```

```

PS C:\Users\manue\Desktop\elixir> elixir cajero.exs
Ingrese su PIN: 0482
Tipo de cuenta (ahorros/corriente): ahorros
Monto a retirar: 450000
Error: Fondos insuficientes para cubrir el monto y la comisión
PS C:\Users\manue\Desktop\elixir> elixir cajero.exs
Ingrese su PIN: 0482
Tipo de cuenta (ahorros/corriente):
Monto a retirar: 95000
Error: Tipo de cuenta no válido
PS C:\Users\manue\Desktop\elixir> elixir cajero.exs
Ingrese su PIN: 0482
Tipo de cuenta (ahorros/corriente): ahorros
Monto a retirar: 95000
Error: El monto debe ser múltiplo de $10.000
PS C:\Users\manue\Desktop\elixir> elixir cajero.exs
Ingrese su PIN: 1234
Tipo de cuenta (ahorros/corriente): ahorros
Monto a retirar: 700000
Error: PIN incorrecto
PS C:\Users\manue\Desktop\elixir> elixir cajero.exs
Ingrese su PIN: 0482
Tipo de cuenta (ahorros/corriente): nomina
Monto a retirar: 50000
Error: Tipo de cuenta no válido
PS C:\Users\manue\Desktop\elixir> 

```

2. **El PIN como entero.** Explique qué pasaría si el PIN se leyera con `Util.leer(mensaje, :integer)`. Pruébalo con el PIN 0482 y con el texto abcd. ¿Por qué el segundo caso es especialmente grave?

R= Si el pin se lee como número entero, el valor `"0482"` pierde el cero inicial y se convierte en `482`, haciendo que la validación falle porque no coincide con la clave de 4 dígitos. El caso de `"abcd"` es especialmente grave porque ingresar letras donde se espera un número provoca (`ArgumentError`) que rompe y detiene por completo la ejecución del programa en la consola.

3. **Orden de las validaciones.** ¿Por qué la validación del PIN debe ir antes que la del saldo? Piense en qué información obtendría alguien que no conoce el PIN si el orden fuera al revés.

R= La validación del PIN debe ir siempre de primero por razones de seguridad, ya que si se validara antes el saldo, un extraño sin la clave podría adivinar cuánto dinero tiene la cuenta haciendo pruebas con distintos montos

4. **Retiro de \$450.000.** En el caso de ahorros con \$450.000 el monto sí cabe en el saldo, pero el retiro se rechaza. Explique por qué y opine si el mensaje de error que muestra su programa es suficientemente claro para el cliente.

R=Aunque el usuario tiene \$450.000 en su saldo y pide retirar \$450.000, la transacción se rechaza porque las cuentas de ahorro cobran una comisión de \$2.500 cuando el retiro supera los \$200.000, lo que da un cobro total de \$452.500 que sobrepasa el dinero disponible. El mensaje **"Fondos insuficientes para cubrir el monto y la comisión"** es claro para el cliente porque explica de forma transparente que el rechazo se debe a la suma de la comisión y no solo al retiro.

5. **Menor cantidad de billetes.** Demuestre con dos ejemplos que su función entrega la menor cantidad de billetes. ¿Seguiría funcionando si el cajero tuviera billetes de \$50.000, \$30.000 y \$20.000? Pruebe con \$60.000.

R= La función garantiza entregar la menor cantidad de billetes al dividir siempre por la denominación más grande primero: para \$150.000 entrega solo 3 billetes de \$50.000 (en vez de 15 de \$10.000) y para \$80.000 entrega 3 billetes (uno de \$50.000, uno de \$20.000 y uno de \$10.000). Sin embargo, el programa **no** funcionaría correctamente si existieran billetes de \$30.000, ya que para retirar \$60.000 el sistema entregaría 1 billete de \$50.000 y fallaría al no poder dar el resto de \$10.000, en lugar de entregar la solución correcta de 2 billetes de \$30.000.

Punto 4: Depuración

La biblioteca de la universidad cobra una multa por devolución tardía con estas reglas:

- **Valor de la multa:** \$500 por día de retraso, con un tope de \$20.000 a partir de 41 días.
- **Descuento:** los estudiantes pagan la mitad y los docentes pagan completo.
- **Código del libro:** debe tener 6 caracteres.
- **Mensaje sin multa:** si no hay multa, se muestra "no tiene multa".

Un compañero escribió el siguiente programa, pero **contiene al menos seis errores**. Algunos impiden compilar, otros hacen fallar el programa en ejecución y otros producen resultados incorrectos sin fallar.

```
defmodule Biblioteca do
```

```

@moduledoc """
Calcula la multa por devolución tardía de un libro.
"""

@valor_dia 500
@tope_multa 20000

def main do

  codigo = Util.leer("Ingrese el código del libro: ", :string)
  dias = Util.leer("Ingrese los días de retraso: ", :integer)
  tipo = Util.leer("Tipo de usuario (estudiante/docente): ", :string)

  procesar(codigo, dias, tipo)
  |> generar_mensaje()
  |> Util.imprimir_mensaje()

end

defp procesar(codigo, dias, tipo) do

  with {:ok, codigo} <- validar_codigo(codigo),
       {:ok, tipo} <- convertir_tipo(tipo) do

    multa =
      calcular_multa(dias)
      |> aplicar_descuento(tipo)

    {:ok, codigo, multa}

  end

end

defp validar_codigo(codigo) when String.length(codigo) == 6, do: {:ok, codigo}
defp validar_codigo(_), do: {:error, "El código debe tener 6 caracteres"}

defp convertir_tipo("estudiante"), do: {:ok, :estudiante}
defp convertir_tipo("docente"), do: :docente
defp convertir_tipo(_), do: {:error, "Tipo de usuario no válido"}

defp calcular_multa(dias) when dias > 0, do: dias * @valor_dia
defp calcular_multa(dias) when dias > 40, do: @tope_multa
defp calcular_multa(_), do: 0

defp aplicar_descuento(:estudiante, multa), do: multa - multa * 0.5
defp aplicar_descuento(:docente, multa), do: multa

defp generar_mensaje({:ok, codigo, 0}), do: "El libro #{codigo} no tiene multa"

```

```

defp generar_mensaje({:ok, codigo, multa}), do: "El libro #{codigo} tiene una multa de
#{multa}"
defp generar_mensaje({:error, motivo}), do: "Error: #{motivo}"

end

Biblioteca.main()

```

Actividades:

Predicción. Antes de ejecutar, lea el código y escriba qué errores cree que tiene. viendo el código el primer error que se identifica es en calcular multa el orden de las restricciones que están trocadas y nunca va a llegar a el tope primero se debe de validar días>40 y no días>0.

También en el procesar se esta validando tipo y código pero falta validar los días with{ok,dias}

haciendo la comparacion de los errores con la IA quedan faltando 4 errores que serian

`convertir_tipo("docente")` Devuelve `:docente` en vez de `{:ok, :docente}`: retorno inconsistente con las otras cláusulas .

`aplicar_descuento/2` Parámetros invertidos respecto a cómo se invoca en el pipe .

`aplicar_descuento estudiante* 0.5` produce float: `$2500.0`, y el `0.0` ya no coincide con la cláusula `{:ok, codigo, 0}` .

`with` sin `else` Funciona "por accidente" con `{:error, motivo}`, pero deja pasar cualquier otro valor no coincidente (E1) hasta `generar_mensaje/1`

Ejecución. Ejecute el programa y corrija los errores uno por uno. Por cada error registre:

- El mensaje exacto que mostró Elixir, si lo hubo.
- Su tipo: compilación, ejecución o lógica.
- La causa, explicada con los conceptos de las guías.
- La corrección aplicada.
- Los datos de entrada que permiten detectarlo.

3.Errores ocultos. Explique por qué algunos errores no se pueden ver hasta corregir otros.

-cuando se está ejecutando y se identifica un error la ejecución se detiene al identificar la primer excepción como aplicar descuento se ve antes de generar mensaje por lo cual antes de que se imprima los errores no tienen como manifestarse

-en el with al convertir_tipo con docente no se ejecuta calcular_multa ni aplicar_descuento y donde viene el tope de multa queda sin recorrer

-error de lógica que solo se puede identificar al correr el programa y dándose cuenta que la salida no es la esperada como son los errores 3 y 4

4.Días inválidos. Agregue una validación de días dentro del with: los días no pueden ser negativos. Discuta qué pasa hoy si el usuario escribe -5 o abc.

-Si el usuario escribe -5 , esto no cumple ninguna restricción así que entra y devuelve 0,el programa sirve como si no hubiera deuda .en pocas palabras una entrada inválida se acepta y se interpreta como válida .

-Si el usuario

Verificación. Entregue el código corregido y la salida de estos casos:

- ABC123, 10, estudiante=El libro ABC123 tiene una multa de \$2500
- ABC123, 50, estudiante=El libro ABC123 tiene una multa de \$10000
- ABC123, 0, estudiante=El libro ABC123 no tiene multa
- ABC123, 50, docente=El libro ABC123 tiene una multa de \$20000
- AB12, 5, docente=Error: El código debe tener 6 caracteres

```
defmodule Biblioteca do

  @moduledoc """
  Calcula la multa por devolución tardía de un libro.
  """

  @valor_dia 500
  @tope_multa 20000
  @descuento_estudiante 50

  def main do

    codigo = Util.leer("Ingrese el código del libro: ", :string)
    dias = Util.leer("Ingrese los días de retraso: ", :integer)
    tipo = Util.leer("Tipo de usuario (estudiante/docente): ", :string)

    procesar(codigo, dias, tipo)
    |> generar_mensaje()
    |> Util.imprimir_mensaje()

  end

  defp procesar(codigo, dias, tipo) do

    with {:ok, codigo} <- validar_codigo(codigo),
```

```

        {:ok, dias} <- validar_dias(dias),
        {:ok, tipo} <- convertir_tipo(tipo) do

    multa =
        calcular_multa(dias)
        |> aplicar_descuento(tipo)

    {:ok, codigo, multa}

end

end

defp validar_codigo(codigo) do
  if String.length(codigo) == 6 do
    {:ok, codigo}
  else
    {:error, "El código debe tener 6 caracteres"}
  end
end

defp validar_dias(dias) when is_integer(dias) and dias >= 0, do: {:ok, dias}
defp validar_dias(_), do: {:error, "Los días de retraso no pueden ser negativos"}

defp convertir_tipo("estudiante"), do: {:ok, :estudiante}
defp convertir_tipo("docente"), do: {:ok, :docente}
defp convertir_tipo(_), do: {:error, "Tipo de usuario no válido"}

defp calcular_multa(dias) when dias > 40, do: @tope_multa
defp calcular_multa(dias) when dias > 0, do: dias * @valor_dia
defp calcular_multa(_), do: 0

defp aplicar_descuento(multa, :estudiante), do: div(multa * @descuento_estudiante,
100)
defp aplicar_descuento(multa, :docente), do: multa

defp generar_mensaje({:ok, codigo, 0}), do: "El libro #{codigo} no tiene multa"
defp generar_mensaje({:ok, codigo, multa}), do: "El libro #{codigo} tiene una multa
de #{multa}"
defp generar_mensaje({:error, motivo}), do: "Error: #{motivo}"

```

```
end
```

```
Biblioteca.main()
```

Tabla de errores:

Número	Tipo	Mensaje de Elixir (si lo hubo)	Causa de la falla	Corrección aplicada	Datos para detectarlo
1	Compilación	invalid call to String.length/1 in guard	No se permiten herramientas complejas de texto al lado de la palabra `when`.	Se cambió `String.length` por la herramienta básica <code>if</code> .	Ocurre al intentar ejecutar cualquier caso (el programa no arranca).
2	Ejecución	no with clause matching: :docente	La estructura `with` es estricta y esperaba un `{ :ok, ... }`, pero el código solo entregaba `:docente`.	Se empacó la respuesta correctamente: `{ :ok, :docente }`.	ABC123, 5, docente
3	Ejecución	no function clause matching in...	El tubo (` >`) entrega el paquete como primer dato, pero la función lo pedía de segundo.	Se voltearon los parámetros en la función: `aplicar_descuento(multa, tipo)`.	ABC123, 5, estudiante
4	Lógica	Falla en silencio	Elixir lee de arriba a abajo. La regla "mayor a 0" atrapaba todos los números y bloqueaba el tope de 40 días.	Se movió la regla del tope (`> 40`) para que quede por encima de la otra.	ABC123, 50, docente
5	Lógica	Falla en silencio	Multiplicar por `0.5` convierte la plata en decimales. Elixir esperaba dinero en números enteros.	Se cambió la multiplicación por división entera: `div(multa, 2)`.	ABC123, 0, estudiante
6	Lógica	Imprimía "multa de 0.0"	Consecuencia del error 5. Como daba decimal (`0.0`), no lo reconocía como el cero exacto para imprimir "no tiene multa".	Se arregló automáticamente al usar la división entera del error 5.	ABC123, 0, estudiante

Si le pide a una IA que corrija el código, compare su respuesta con su propia lista. Indique qué errores encontró la IA, cuáles no y si alguna "corrección" introdujo conceptos no vistos.

Entrega

- Un archivo .exs por punto y el util.exs utilizado.

- Un documento con:
 - Las respuestas de análisis.
 - Las tablas de casos de prueba con la salida real.
 - La tabla de errores del Punto 4.
 - La bitácora de uso de IA.